DISEÑO COMPUTACIONAL · SUB-ETAPA 7 · BASE DE DATOS

Modelo Relacional

Las tablas derivadas del MER: llaves, acciones referenciales, normalización y el mecanismo de cada restricción de integridad, probado contra MySQL 8.4.

PROYECTO

AdaptiveLearn

CURSO

DSE-2 · Diseño y Desarrollo de Software Educativo II

METODOLOGÍA

MODESEC · Fase III

FECHA

22 de octubre de 2026

Metodología: MODESEC — Fase III, Diseño Computacional, modelado de la base de datos. **Fecha:** 2026-10-21. **Archivo generado** por `modelo-relacional/documento_mr.py` desde `modelo-relacional/datos_mr.py`; no se edita a mano.

Actualizado el 2026-10-26 por la revisión del docente del 2026-10-22: tablas `area`, `grado` y `documento_oficial`, código del DBA `ÁREA-GG-NN` (ADR-036) y `credencial.estado_verificacion` (ADR-037). Ver la sección 5.6 y la tabla de cambios (10).

MODESEC pide *«un MR (Modelo relacional) basado en el MER»*. La base es `16-modelo-entidad-relacion.md`; `modelo-relacional/_verificar_mr_mer.py` comprueba en cada corrida que este modelo no se aparte de él, entidad por entidad y atributo por atributo.

Motor: MySQL 8 (RNF-14, *«NestJS y MySQL»*). El esquema ejecutable es `modelo-relacional/mr_mysql.sql`, y **se probó contra MySQL real** —8.4 y 8.0— (sección 9). No usa nada que MariaDB 10.11 no tenga —el backend de I1 hoy corre sobre MariaDB—, pero en MariaDB no se ejecutó: queda como prueba pendiente al integrar.

---

1. Resumen

Tablas	Columnas	Llaves foráneas	UNIQUE	CHECK
18	104	23	17	14

Una tabla por entidad del MER; Reporte no tiene tabla (sección 5). Los tipos, tamaños y dominios de cada columna están en `18-diccionario-datos.md`.

---

2. Esquema

Llave primaria <u>subrayada</u>; llave foránea en *cursiva*, con la tabla a la que apunta.

- **credencial**(<u>id_credencial</u>, *id_docente* → docente, *id_estudiante* → estudiante, correo, proveedor, password_hash, proveedor_id, estado_verificacion, codigo_verificacion, codigo_expira_en, codigo_ultimo_envio, intentos_codigo, intentos_login, bloqueado_hasta, creado_en, actualizado_en)
- **docente**(<u>id_docente</u>, nombres, apellidos, avatar_url)
- **estudiante**(<u>id_estudiante</u>, nombres, apellidos, avatar_url)
- **experiencia**(<u>id_experiencia</u>, *id_docente* → docente, *codigo_dba* → dba, nombre, estado, fecha_creacion, fecha_publicacion, codigo_invitacion, servicio_ia, modelo_ia)
- **area**(<u>codigo_area</u>, nombre)
- **grado**(<u>numero</u>, nombre, nivel)
- **documento_oficial**(<u>id_documento</u>, *codigo_area* → area, titulo, version, anio, paginas, url_oficial)
- **dba**(<u>codigo_dba</u>, *codigo_area* → area, documento_oficial, *numero_grado* → grado, *id_documento* → documento_oficial, enunciado, ejemplo)
- **evidencia**(*<u>codigo_dba</u>* → dba, <u>numero_evidencia</u>, enunciado)
- **evidencia_dictamen**(*<u>codigo_dba</u>* → evidencia, *<u>numero_evidencia</u>* → evidencia, aplica_formato_cerrado, formato, justificacion, fecha_dictamen)

- **experiencia_evidencia**(*<u>id_experiencia</u>* → experiencia, *<u>codigo_dba</u>* → experiencia, evidencia, *<u>numero_evidencia</u>* → evidencia)
- **inscripcion**(<u>id_inscripcion</u>, *id_estudiante* → estudiante, *id_experiencia* → experiencia, fecha_union)
- **item**(<u>id_item</u>, *id_experiencia* → experiencia_evidencia, *codigo_dba* → experiencia_evidencia, *numero_evidencia* → experiencia_evidencia, enunciado, formato, origen, excluido)
- **opcion**(<u>id_opcion</u>, *id_item* → item, texto, orden, es_correcta, grupo, hipotesis_error)
- **presentacion**(*<u>id_inscripcion</u>* → inscripcion, *<u>id_item</u>* → item, *id_experiencia* → inscripcion, item, orden_presentacion, fecha_presentacion)
- **respuesta**(<u>id_respuesta</u>, *id_inscripcion* → presentacion, *id_item* → presentacion, certeza, fecha_respuesta, correcta)
- **respuesta_opcion**(*<u>id_respuesta</u>* → respuesta, *<u>id_opcion</u>* → opcion, *id_item* → respuesta, opcion, valor_dado)
- **resultado**(*<u>id_inscripcion</u>* → inscripcion, *id_experiencia* → inscripcion, experiencia_evidencia, *<u>codigo_dba</u>* → experiencia_evidencia, *<u>numero_evidencia</u>* → experiencia_evidencia, estado, fecha_calculo)

Diagrama del esquema: pegar `modelo-relacional/mr_adaptivelearn.dbml` en dbdiagram.io.

---

3. Convención de nombres

- Tablas y columnas en `snake_case`, en singular y sin tildes: `inscripcion`, `opcion`.
- Llaves sustitutas `id_<tabla>`, `INT UNSIGNED AUTO_INCREMENT`. Las llaves naturales son las del catálogo: `codigo_dba`, `codigo_area` y el `numero` del grado (ADR-036).
- Una llave foránea se llama igual que la columna a la que apunta, con una excepción: `dba.numero_grado` apunta a `grado.numero`, para no confundirla con el número del DBA.
- Restricciones con nombre: `fk_<tabla>_<n>`, `uq_<tabla>_<n>`, `ck_<tabla>_<n>`. Un error del motor dice qué regla se rompió.

---

4. Llaves foráneas y política de borrado

Esto cierra el vacío V-11 que el análisis de requerimientos dejó abierto en RF-02-07 (eliminar una experiencia) y RF-04-12 (salir de una): borrar una experiencia o una inscripción borra **en cascada** todo lo que depende de ella. Las raíces que ningún flujo borra —credencial, docente, estudiante, el catálogo (área, grado, documento, DBA) y evidencia— quedan en `RESTRICT`, como seguro. `ON UPDATE CASCADE` en todas, salvo las de `credencial` y las del catálogo hacia `area`, `grado` y `documento_oficial` (abajo).

Tabla	Columnas	Referencia	ON DELETE	ON UPDATE	Por qué
credencial	`id_docente`	`docente(id_docente)`	RESTRICT	RESTRICT	Raíz: ningún flujo borra un docente. Sin ON UPDATE CASCADE: MySQL no admite un CHECK sobre una columna con acción referencial, y aquí lo hay (RI-12).

Tabla	Columnas	Referencia	ON DELETE	ON UPDATE	Por qué
credencial	`id_estudiante`	`estudiante(id_estudiante)`	RESTRICT	RESTRICT	Igual que la anterior.
experiencia	`id_docente`	`docente(id_docente)`	RESTRICT	CASCADE	Raíz: ningún flujo del sistema borra un docente.
experiencia	`codigo_dba`	`dba(codigo_dba)`	RESTRICT	CASCADE	Raíz: dato del MEN, no se borra por razones de este sistema.
documento_oficial	`codigo_area`	`area(codigo_area)`	RESTRICT	RESTRICT	Raíz: el catálogo no se borra por razones de este sistema.
dba	`codigo_area`	`area(codigo_area)`	RESTRICT	RESTRICT	Raíz. Sin ON UPDATE CASCADE: el CHECK de RI-21 lee esta columna.
dba	`numero_grado`	`grado(numero)`	RESTRICT	RESTRICT	Raíz. Sin ON UPDATE CASCADE, por la misma razón.
dba	`id_documento, codigo_area`	`documento_oficial(id_documento, codigo_area)`	RESTRICT	RESTRICT	Compuesta con codigo_area: el documento del que se copió el DBA es de su misma área (RI-20, ciclo C6 del MER).
evidencia	`codigo_dba`	`dba(codigo_dba)`	CASCADE	CASCADE	Entidad débil: sin su DBA no tiene identidad.
evidencia_dictamen	`codigo_dba, numero_evidencia`	`evidencia(codigo_dba, numero_evidencia)`	CASCADE	CASCADE	Entidad débil: el dictamen no existe sin la evidencia dictaminada.
experiencia_evidencia	`id_experiencia, codigo_dba`	`experiencia(id_experiencia, codigo_dba)`	CASCADE	CASCADE	Compuesta con codigo_dba: la evidencia incluida tiene que ser del DBA al que se ancla la experiencia (RI-18, ciclo C1 del MER). Eliminar la experiencia elimina sus evidencias incluidas (V-11).
experiencia_evidencia	`codigo_dba, numero_evidencia`	`evidencia(codigo_dba, numero_evidencia)`	RESTRICT	CASCADE	Hacia el currículo del MEN: no se borra una evidencia oficial en uso.
inscripcion	`id_estudiante`	`estudiante(id_estudiante)`	RESTRICT	CASCADE	Raíz.
inscripcion	`id_experiencia`	`experiencia(id_experiencia)`	CASCADE	CASCADE	Eliminar la experiencia elimina a sus inscritos de ella (V-11, RF-02-07).
item	`id_experiencia, codigo_dba, numero_evidencia`	`experiencia_evidencia(id_experiencia, codigo_dba, numero_evidencia)`	CASCADE	CASCADE	UNA sola FK compuesta: el ítem es de una evidencia que su experiencia incluye.
opcion	`id_item`	`item(id_item)`	CASCADE	CASCADE	Composición: una opción no existe sin su ítem.

Tabla	Columnas	Referencia	ON DELETE	ON UPDATE	Por qué
presentacion	`id_inscripcion, id_experiencia`	`inscripcion(id_inscripcion, id_experiencia)`	CASCADE	CASCADE	Salir de la experiencia borra lo que se le presentó (V-11, RF-04-12). La columna compartida id_experiencia hace estructural RI-13.
presentacion	`id_item, id_experiencia`	`item(id_item, id_experiencia)`	CASCADE	CASCADE	El ítem presentado es de la misma experiencia (RI-13). CASCADE y no RESTRICT: un ítem solo desaparece cuando desaparece su experiencia entera (ADR-026: descartar no borra), y eso debe poder hacerse aunque tenga respuestas (RF-02-07).
respuesta	`id_inscripcion, id_item`	`presentacion(id_inscripcion, id_item)`	CASCADE	CASCADE	Una respuesta sin presentación previa es imposible por estructura (H-3).
respuesta_opcion	`id_respuesta, id_item`	`respuesta(id_respuesta, id_item)`	CASCADE	CASCADE	Sin la respuesta, la selección no tiene sentido. Compuesta con id_item: fija de qué ítem es la respuesta (RI-6).
respuesta_opcion	`id_opcion, id_item`	`opcion(id_opcion, id_item)`	CASCADE	CASCADE	Compuesta con id_item: la opción elegida es del ítem que se presentó (RI-6, ciclo C4). CASCADE y no RESTRICT: una opción solo se borra editando un ítem en Borrador, cuando todavía no hay respuestas (ADR-026, RI-10), o al eliminar la experiencia entera, que debe poder hacerse (RF-02-07).
resultado	`id_inscripcion, id_experiencia`	`inscripcion(id_inscripcion, id_experiencia)`	CASCADE	CASCADE	La experiencia del resultado es la de su inscripción (RI-13). Salir borra los resultados (V-11).
resultado	`id_experiencia, codigo_dba, numero_evidencia`	`experiencia_evidencia(id_experiencia, codigo_dba, numero_evidencia)`	CASCADE	CASCADE	La evidencia medida es una que esa experiencia incluye. CASCADE para que eliminar la experiencia no quede bloqueado por sus resultados (V-11).

Por qué las de `credencial` no llevan `ON UPDATE CASCADE`: MySQL no admite una restricción CHECK sobre una columna que participa en una acción referencial, y `id_docente` / `id_estudiante` tienen el CHECK de RI-12. Como son llaves sustitutas que nunca se actualizan, `RESTRICT` no cuesta nada. **Lo mismo en las del catálogo:** el CHECK de RI-21 lee `dba.codigo_area` y `dba.numero_grado`, y un código del catálogo no cambia.

Lo que la cascada nunca toca: una cuenta, ni el currículo. La prueba de la sección 9 elimina una experiencia con un inscrito que respondió y cuenta, tabla por tabla, lo que queda.

5. Decisiones de este modelo

5.1 `id_experiencia` en `presentacion` y en `resultado` (RI-13)

El MER dejó esta decisión al modelo relacional: *«que esa deducción se resuelva con una columna redundante o con una regla, lo decide el Modelo Relacional»*. Se resuelve con la columna, en las dos tablas. `id_experiencia` participa en **dos llaves foráneas compuestas** a la vez —hacia `inscripcion(id_inscripcion, id_experiencia)` y hacia `item(id_item, id_experiencia)` en `presentacion`; hacia `inscripcion` y hacia `experiencia_evidencia` en `resultado`—, así que el motor rechaza una presentación de un ítem de otra experiencia, o un resultado de una evidencia que la experiencia no incluye, sin triggers. Para que esas FK sean posibles, `inscripcion` e `item` declaran un UNIQUE sobre `(id, id_experiencia)`.

El borrador del MR había resuelto esto solo en `resultado` y dejó declarado el hueco en `presentacion`. Ya no está: la batería de pruebas intenta las dos combinaciones posibles de presentar un ítem ajeno y el motor rechaza ambas.

5.2 Los ciclos C1 y C4 del MER, cerrados con la misma técnica (RI-18, RI-6)

El MER declara sus seis ciclos y qué cierra cada uno (`16`, sección 6). Dos estaban abiertos, y el motor aceptaba los datos que los rompen; las pruebas de `pruebas_mr.py` lo demostraron:

- **C1, RI-18.** Una experiencia podía incluir una evidencia de **otro DBA**. Ahora la FK de `experiencia_evidencia` hacia `experiencia` es compuesta, `(id_experiencia, codigo_dba)`, y `experiencia` declara `UNIQUE(id_experiencia, codigo_dba)` para poder ser su destino.
- **C4, RI-6.** Una respuesta podía elegir una opción de **otro ítem**. Ahora `respuesta_opcion` lleva `id_item` dentro de sus dos FK compuestas —hacia `respuesta(id_respuesta, id_item)` y hacia `opcion(id_opcion, id_item)`—, igual que `id_experiencia` en `presentacion`.

5.3 El servicio de IA se guarda en la experiencia, no en cada ítem (RI-17)

Un banco sale entero de un solo servicio (RF-03-02). Guardar `servicio_ia` y `modelo_ia` en cada ítem repetía el mismo valor una vez por ítem y, peor, dejaba que un banco mezclara servicios. Van en `experiencia`, una sola vez y `NOT NULL` (toda experiencia tiene banco: el «todo o nada» de ADR-027). El servicio de un ítem de origen `ia` o `editado_docente` es el de su experiencia; uno `creado_docente` no se atribuye a la IA. Así RF-03-03 —saber qué servicio y modelo generó cada ítem— se cumple sin repetir el dato.

5.4 Docente y Estudiante, cada uno con su tabla; Usuario sin tabla

El diagrama de clases generaliza a los dos en `Usuario` (abstracta). En la base, cada subclase concreta tiene su tabla con los atributos heredados: no hay usuarios sin rol, y Docente y Estudiante son tipos separados (RF-01-06). La credencial apunta a uno **o** al otro (CHECK de RI-12), y el correo, en `credencial`, es único entre los dos.

5.5 Reporte no es una tabla

Se calcula sobre `resultado` cada vez que el docente lo abre (ADR-022): estado general por estudiante y clasificación por evidencia. Guardarlo crearía un segundo lugar donde el mismo dato podría quedar viejo.

5.6 El catálogo normalizado: ciclo C6 y código del DBA (ADR-036, RI-20, RI-21)

- **C6, RI-20.** El documento del que se copió un DBA tiene que ser de su misma área. Se cierra con la técnica de C1 y C4: la FK de `dba` hacia `documento_oficial` es compuesta, `(id_documento, codigo_area)`, y `documento_oficial` declara `UNIQUE(id_documento, codigo_area)` para poder ser su destino. La prueba «un DBA de Matemáticas copiado del documento de Lenguaje» lo intenta y el motor lo rechaza.
- **RI-21.** Un CHECK arma el código esperado con el área y el grado de la misma fila y lo compara con `codigo_dba`: `MAT-08-01` solo entra con `codigo_area = 'MAT'` y `numero_grado = 8`. Así el código no puede contradecir las llaves foráneas.
- **Decisión de esta sesión, para que el equipo la confirme:** las columnas de código (`codigo_dba`, `codigo_area` y sus copias en otras tablas) son `CHARACTER SET ascii COLLATE ascii_bin`. ADR-036 pide códigos sin tildes; con ASCII, el motor ni siquiera puede guardar una tilde, y con el cotejamiento binario `mat` no es igual a `MAT`. El resto de la base sigue en `utf8mb4_unicode_ci`.
- **Defecto conocido de los datos, no del modelo.** `curriculo_men_normalizado.json` tiene 342 DBA y solo 329 códigos distintos (16 §4, DBA). `codigo_dba` sigue siendo llave primaria: cargar hoy el JSON completo **fallaría por llave duplicada** en esos 13, y eso es lo correcto. Se corrige en la auditoría del currículo (pendientes, A12).

---

6. Análisis de normalización

Todas las tablas están en tercera forma normal, con tres excepciones deliberadas y controladas. En las de llave sustituta simple no hay dependencias parciales posibles; se revisó que ningún atributo no clave dependa de otro no clave. En las de llave compuesta (`evidencia`, `evidencia_dictamen`, `experiencia_evidencia`, `presentacion`, `respuesta_opcion`, `resultado`), cada atributo no clave depende de la llave entera, salvo:

Dónde	Dependencia	Por qué se acepta
`presentacion.id_experiencia`, `resultado.id_experiencia`, `respuesta_opcion.id_item`	Dependen solo de una parte de la llave (`id_inscripcion`, `id_respuesta`): dependencia parcial, que viola la 2FN en sentido estricto.	Es la redundancia de 5.1 y 5.2. No puede quedar inconsistente: las FK compuestas la obligan a coincidir. A cambio, RI-13 y RI-6 son estructurales.
`experiencia.modelo_ia` → `experiencia.servicio_ia`	Un modelo pertenece a un solo servicio (dependencia transitiva).	Los dos son el registro de lo que ocurrió al generar el banco (RF-03-03), no datos que se editen. Normalizar exigiría una tabla de modelos sin ningún otro uso. Pasarlos de `item` a `experiencia` (5.3) ya quitó la repetición mayor.
`respuesta.correcta`, `resultado.estado`	Derivados de otras filas.	ADR-021: se guardan a propósito, como fotografía del momento (RNF-11). RI-11 exige que coincidan con el recálculo.

7. Restricciones de integridad: quién hace cumplir cada una

Una restricción que la base puede hacer cumplir, la hace cumplir la base. Las demás dependen de datos de otra tabla —un CHECK de MySQL no puede consultarlas— o del momento en que ocurre algo, y las hace cumplir el servicio de la capa NestJS que se nombra; no se usan triggers, para que cada regla de negocio viva en un solo lugar y se pueda probar.

RI	Mecanismo	Dónde
RI-1	Motor: `UNIQUE(id_estudiante, id_experiencia)`	`inscripcion`
RI-2	Motor: la llave primaria de la entidad débil	`resultado`
RI-3	Motor: `UNIQUE(correo)` con cotejamiento sin mayúsculas; `UNIQUE(codigo_invitacion)` y CHECK del alfabeto	`credencial`, `experiencia`
RI-4	Servicio: depende del formato, que está en otra tabla	`Item.esValido()` (CU-16, CU-18, CU-20)
RI-5	Servicio: al crear la experiencia, solo entran las evidencias admitidas	`DBA.evidenciasAdmitidas()` (CU-11, CU-16)
RI-6	Motor: que la opción sea del ítem presentado, con `id_item` en las dos FK compuestas de `respuesta_opcion` (sección 5.2). Servicio: al menos una opción, y exactamente una en selección única	`respuesta_opcion` · `ServicioDiagnostico.responder()`, `Item.evaluar()` (CU-24)
RI-7	Servicio: al unirse	`ServicioDiagnostico.unirse()` (CU-22)
RI-8	Motor: `UNIQUE(id_inscripcion, orden_presentacion)` y `CHECK(orden_presentacion > 0)`	`presentacion`
RI-9	Servicio: la regla no presenta más de 6	`ReglaAdaptativa` (CU-25)
RI-10	Servicio: el banco solo se edita en Borrador	`Experiencia.bancoEditable()` (CU-17 a CU-20)
RI-11	Servicio: se calcula al escribir; verificable recalculando	`Item.evaluar()`, `ReglaAdaptativa.concluir()`
RI-12	Motor: `CHECK(id_docente IS NULL OR id_estudiante IS NULL)` y UNIQUE sobre cada una	`credencial`
RI-13	Motor: llaves foráneas compuestas con `id_experiencia` (sección 5.1)	`presentacion`, `resultado`
RI-14	Servicio: al publicar, se comprueba otra vez	`Experiencia.puedePublicarse()` (CU-13)
RI-15	Motor: CHECK, con `estado_verificacion = 'verificado'` (ADR-037)	`credencial`
RI-16	Motor: CHECK	`experiencia`
RI-17	Motor: un solo `servicio_ia` y un solo `modelo_ia`, NOT NULL, por experiencia: un banco no puede mezclar servicios	`experiencia`

RI	Mecanismo	Dónde
RI-18	Motor: FK compuesta `(id_experiencia, codigo_dba)` de `experiencia_evidencia` hacia `experiencia` (sección 5.2)	`experiencia_evidencia`
RI-20	Motor: FK compuesta `(id_documento, codigo_area)` de `dba` hacia `documento_oficial` (sección 5.6)	`dba`
RI-21	Motor: CHECK que arma `ÁREA-GG-NN` con el área y el grado de la fila; CHECK del formato de `codigo_area`; columnas ASCII binarias (sección 5.6)	`dba`, `area`
RI-19	Motor: CHECK de que el dictamen que admite formato cerrado diga cuál. Servicio: el ítem usa el formato del dictamen al generarse y al crearse	`evidencia_dictamen` · `GeneradorBanco`, `ServicioBanco.agregarItem()` (CU-16, CU-20)

Restricciones de dominio (CHECK) — expresión de MySQL:

- `credencial`: `id_docente IS NULL OR id_estudiante IS NULL` — una credencial accede como un solo rol. *RI-12 · 20-spec-auth-elige-rol.md*
- `credencial`: `(proveedor = 'correo' AND password_hash IS NOT NULL AND proveedor_id IS NULL) OR (proveedor <> 'correo' AND password_hash IS NULL AND proveedor_id IS NOT NULL AND estado_verificacion = 'verificado')` — contraseña solo con correo; identificador externo y correo verificado solo con Google o Microsoft. *RI-15 · RF-01-01 · RF-01-02*
- `docente`: `TRIM(nombres) <> '' AND TRIM(apellidos) <> ''` — nombres y apellidos no vacíos. *RF-01-05*
- `estudiante`: `TRIM(nombres) <> '' AND TRIM(apellidos) <> ''` — nombres y apellidos no vacíos. *RF-01-05*
- `experiencia`: `(estado = 'Borrador' AND fecha_publicacion IS NULL) OR (estado = 'Publicada' AND fecha_publicacion IS NOT NULL)` — fecha de publicación solo si está Publicada. *RI-16 · RF-02-05*
- `experiencia`: `codigo_invitacion REGEXP '^[ABCDEFGHJKMNPQRSTUVWXYZ23456789]{6}$'` — 6 caracteres de letras y números, sin 0 O 1 I L. *RI-3 · RF-02-06*
- `area`: `codigo_area REGEXP '^[A-Z]{2,3}$'` — 2 o 3 letras mayúsculas, sin tildes (en MySQL la columna es ASCII binaria: una tilde no cabe y el REGEXP distingue mayúsculas). *ADR-036*
- `grado`: `(numero BETWEEN 1 AND 5 AND nivel = 'basica_primaria') OR (numero BETWEEN 6 AND 9 AND nivel = 'basica_secundaria') OR (numero BETWEEN 10 AND 11 AND nivel = 'media')` — grados 1 a 11, sin Transición, y el nivel que les da la Ley 115. *ADR-036 · ADR-033 · Ley 115 de 1994*
- `documento_oficial`: `paginas > 0 AND TRIM(titulo) <> '' AND TRIM(version) <> ''` — título y versión no vacíos; al menos una página. *ADR-036*
- `dba`: `codigo_dba = CONCAT(codigo_area, '-', LPAD(numero_grado, 2, '0'), '-', RIGHT(codigo_dba, 2)) AND RIGHT(codigo_dba, 2) REGEXP '^[0-9]{2}$' AND RIGHT(codigo_dba, 2) <> '00'` — el código es ÁREA-GG-NN, con el área y el grado de la fila y un número de dos cifras desde 01. *RI-21 · ADR-036*
- `evidencia`: `numero_evidencia > 0` — posición dentro del DBA. *MER*
- `evidencia_dictamen`: `(aplica_formato_cerrado = 1 AND formato IS NOT NULL) OR (aplica_formato_cerrado = 0 AND formato IS NULL)` — si admite formato cerrado dice cuál; si no, no tiene formato. *RI-19 · RF-03-01*
- `presentacion`: `orden_presentacion > 0` — orden desde 1. *RI-8*
- `respuesta`: `certeza BETWEEN 0 AND 5` — escala CRI. *ADR-008 punto 2*

Además, cada columna `ENUM` restringe su dominio, y el modo estricto de MySQL (activo por defecto) rechaza un valor fuera de él en vez de truncarlo.

UNIQUE:

- `credencial(correo)` — RI-3 · RF-01-04: una cuenta por correo, entre los dos roles
- `credencial(id_docente)` — una credencial por docente (1:1)
- `credencial(id_estudiante)` — una credencial por estudiante (1:1)
- `credencial(proveedor, proveedor_id)` — RF-01-04: una misma cuenta de Google o Microsoft no abre dos credenciales
- `experiencia(codigo_invitacion)` — RI-3
- `area(nombre)` — dos áreas no se llaman igual
- `grado(nombre)` — dos grados no se llaman igual
- `documento_oficial(codigo_area, version)` — una misma versión de un área se carga una sola vez (ADR-033, punto 4)
- `documento_oficial(id_documento, codigo_area)` — destino de FK: habilita la comprobación estructural de RI-20
- `experiencia(id_experiencia, codigo_dba)` — destino de FK: habilita la comprobación estructural de RI-18
- `inscripcion(id_estudiante, id_experiencia)` — RI-1
- `inscripcion(id_inscripcion, id_experiencia)` — destino de FK: habilita la comprobación estructural de RI-13
- `item(id_item, id_experiencia)` — destino de FK: habilita la comprobación estructural de RI-13
- `presentacion(id_inscripcion, orden_presentacion)` — RI-8
- `respuesta(id_inscripcion, id_item)` — una presentación, una respuesta
- `respuesta(id_respuesta, id_item)` — destino de FK: habilita la comprobación estructural de RI-6
- `opcion(id_opcion, id_item)` — destino de FK: habilita la comprobación estructural de RI-6

---

8. Índices

Toda llave primaria y todo UNIQUE ya es un índice. Se agrega uno por cada llave foránea cuyas columnas no encabezan ya alguno de ellos:

- `experiencia(id_docente)`
- `experiencia(codigo_dba)`
- `dba(codigo_area)`
- `dba(numero_grado)`
- `dba(id_documento, codigo_area)`
- `experiencia_evidencia(codigo_dba, numero_evidencia)`
- `inscripcion(id_experiencia)`
- `item(id_experiencia, codigo_dba, numero_evidencia)`
- `opcion(id_item)`
- `presentacion(id_inscripcion, id_experiencia)`

- `presentacion(id_item, id_experiencia)`
- `respuesta_opcion(id_respuesta, id_item)`
- `respuesta_opcion(id_opcion, id_item)`
- `resultado(id_inscripcion, id_experiencia)`
- `resultado(id_experiencia, codigo_dba, numero_evidencia)`

---

9. Verificación mecánica

Cuatro scripts, de menor a mayor fuerza. Desde `fase-3-diseno-computacional/modelo-relacional/`:

```
python generar.py
python _verificar_mr_mer.py
python verificar_mr.py
python probar_mysql.py --cnf RUTA_DEL_ARCHIVO_DE_OPCIONES --mysql RUTA_DE_mysql
```

Script	Qué comprueba
`generar.py`	Escribe el diccionario (18), el DBML y los dos SQL desde la fuente.
`_verificar_mr_mer.py`	Que cada entidad del MER tenga su tabla y cada atributo su columna, con las mismas llaves primarias; que toda FK apunte a una PK o a un UNIQUE; que toda columna que el MER no declara tenga su justificación escrita (`EXTRA_JUSTIFICADAS`).
`verificar_mr.py`	Contra SQLite en memoria: que el esquema cargue, la batería de pruebas de `pruebas_mr.py`, y que el DBML y el SQL describan lo mismo. **80 comprobaciones.**
`probar_mysql.py`	La **misma batería** contra un MySQL real: crea una base de prueba, carga `mr_mysql.sql` tal cual, corre cada prueba dentro de una transacción que se deshace, y borra la base al terminar.

La batería (`pruebas_mr.py`) tiene tres clases de prueba: datos que rompen una regla y el motor *debe rechazar* (57), datos raros pero válidos que *debe aceptar* (7), y borrados tras los que se cuenta qué *queda* en cada tabla (3: eliminar una experiencia, salir de una, descartar un ítem). En MySQL, además, cada rechazo tiene que venir **de la restricción que la prueba nombra** (`MOTIVO_MYSQL`): rechazar por otra razón cuenta como falla.

Resultado del 2026-10-26, contra MySQL 8.0.46 (modo estricto por defecto), con el catálogo normalizado y el estado de verificación: 72 comprobaciones, todas correctas. La salida completa está en `modelo-relacional/salida_probar_mysql.txt`. La versión anterior del esquema se había probado el 2026-10-22 contra MySQL 8.4.11; esta todavía no se corrió en 8.4 ni en MariaDB.

Lo que encontró. La primera corrida contra MySQL mostró que la prueba «resultado cuya experiencia no es la de su inscripción» pasaba **por la razón equivocada**: el motor rechazaba la fila por llave primaria duplicada, no por la FK de RI-13. SQLite no lo podía mostrar, porque no nombra la restricción rota. Se corrigió la prueba y desde entonces cada rechazo se exige con su restricción.

10. Cambios respecto del borrador del MR (2026-09-19)

Cambio	Fuente
Tabla nueva `credencial`, con los contadores de intentos y el bloqueo que el backend de I1 ya usa.	V-8 · MER (Credencial) · backend I1
`docente` / `estudiante`: `nombres` y `apellidos`; sin `correo`.	MER
`item`: `origen` con tres valores; sin `estado_revision` (el único dominio abierto del borrador, V-7).	MER
`experiencia`: `servicio_ia` y `modelo_ia`, los del banco entero (5.3).	RI-17 · RF-03-02
`evidencia_dictamen`: `formato`, con su CHECK.	RI-19 · RF-03-01
`experiencia_evidencia` y `respuesta_opcion`: FK compuestas que cierran los ciclos C1 y C4 del MER (5.2).	RI-18 · RI-6
`experiencia_evidencia`: sin `incluida`.	MER
`presentacion`: `id_experiencia` y FK compuestas — cierra el hueco declarado.	RI-13
Política de borrado en cascada desde `experiencia` e `inscripcion`; `presentacion → item` y `respuesta_opcion → opcion` pasan de RESTRICT a CASCADE, para que eliminar una experiencia con respuestas sea posible (RF-02-07).	V-11
CHECK escritos como expresiones SQL reales, con nombre (antes eran prosa).	—
Motor objetivo MySQL 8 (antes MariaDB 10.11), con un DDL propio y probado.	RNF-14
(2026-10-26) Tablas nuevas `area`, `grado` y `documento_oficial`; `dba` pierde `area` y `grado` como texto y gana `codigo_area`, `numero_grado` e `id_documento`, con una FK compuesta que cierra el ciclo C6 (RI-20). `codigo_dba` pasa a `ÁREA-GG-NN`, con su CHECK (RI-21), en ASCII binario en todas las tablas que lo usan (5.6).	ADR-036 · revisión del docente del 2026-10-22 (R1)
(2026-10-26) `credencial`: `email_verificado` pasa a `estado_verificacion` (`procesando` / `verificado`, por defecto `procesando`); el CHECK de RI-15 usa el estado. `codigo_expira_en` y `codigo_ultimo_envio` siguen, a la espera de la pregunta P2.	ADR-037 · revisión del docente del 2026-10-22 (R3, R4)